

Block-Diffusion OD Planning — Block-Size Sweep (mask kernel, raw)

Status (2026-07-16). All seven block sizes {1, 2, 4, 8, 16, 32, 64} of the **mask kernel** are final: trained and evaluated on the new server (143.248.84.179, 4x RTX 3090, one run per GPU, two waves; 43–54 min training each; tmux sessions completed and closed). Per the current request, **all numbers in this report are raw — no `eval_shortest.refine()` (`refine_es`) and no shortest-path patching (`refine_nx`) anywhere**; hence `hit` = arrival throughout. Predecessor report (blk4-only, with refine variants): `RESULTS_BD_prev.pdf` . **Updated 2026-07-17:** added §4 (failure anatomy — length-calibration mechanism, near-miss structure, invalid-edge anatomy) from post-hoc diagnostics on the saved checkpoints. Code-fidelity audit against the reference `bd3lms` implementation: `BD_CODE_VERIFICATION_{K0,EN}.pdf` .

Setup — identical to `RESULTS_BD_prev.pdf` except for the swept block size. BD3-LM-style block diffusion (semi-AR across blocks, absorbing/[MASK] diffusion within each block, SUBS weighted CE, first-hitting sampling). Transformer denoiser: 6 layers, dim 256, adaLN conditioning (~6.8M params; embedding table varies negligibly with canvas length). Trained 1 epoch = 48.2k iters (bs 32, lr 5e-4) on `porto_v3_normal` (index v3-0.05, 1,390 vertices, 1.93M paths). Canvas per the project length spec `[dst, ori, v1, ..., v_L=dst, END...END | PAD...PAD]` : the END tail inside the dst block is a training target, later PAD blocks are excluded from the loss. Conditioning: `dst + ori` as clean in-context prefix tokens plus the OD embedding through adaLN; condition dropout 0.1; CFG off (`w = 1`). Open-ended generation (`length_mode=open`): blocks are appended until `dst` (hit) or `END` (miss) — no length model.

Evaluation. 1,000 held-out OD pairs per run, open-ended, **raw only**. `hit` = the path reaches `dst` unaided (= arrival, since no post-processing exists in this report); `validhit` = usable-path rate; EM / PC / DTW / len metrics as in the companion reports. *Caveat: each run draws its own random 1,000 evaluation pairs (dataset order is shuffled unseeded at load), so cross-run differences of ~±1%p are within sampling noise.*

1. Block-size sweep — mask kernel, raw (no refine)

block size	hit (=arrival)	valid	validhit	EM	PC	DTW	len err	len corr	gen len	real len	s/path
1	0.997	0.943	0.943	0.845	0.883	0.030	0.69	+0.878	23.5	23.0	0.012
2	0.924	0.809	0.777	0.755	0.778	0.044	0.49	+0.977	23.3	23.2	0.004
4	0.943	0.853	0.821	0.789	0.816	0.035	0.32	+0.995	23.7	23.6	0.004
8	0.953	0.807	0.777	0.729	0.761	0.041	0.35	+0.995	22.9	22.9	0.004
16	0.912	0.633	0.598	0.554	0.584	0.062	0.64	+0.979	23.0	23.0	0.004
32	0.900	0.526	0.488	0.443	0.476	0.063	0.69	+0.992	23.4	23.4	0.005
64	0.877	0.504	0.454	0.400	0.439	0.069	0.88	+0.986	24.1	24.0	0.007

Bold = best in column. s/path: first-hitting uses ~1 forward per generated token at every block size, so timing is nearly flat; blk1's 3x overhead comes from the per-block outer loop (append → denoise → scan once per token).

2. Arrival by real path length (raw) — the horizon test

block size	len 2–15	len 16–25	len 26–35	len 36+
1	0.992	1.000	0.996	1.000
2	0.844	0.941	0.958	0.958
4	0.864	0.953	0.978	0.970
8	0.926	0.956	0.961	0.982
16	0.865	0.919	0.931	0.943
32	0.876	0.871	0.934	0.957
64	0.873	0.892	0.884	0.833

3. Findings

- Smaller blocks win, and blk1 (the AR limit) wins outright.** Every quality metric is monotone-ish in favor of small blocks: arrival 0.997 → 0.877 and validity 0.943 → 0.504 as the block grows 1 → 64. blk1 solves the benchmark essentially perfectly without any refinement — 99.7% of paths reach the destination unaided and 94.3% are simultaneously edge-valid.

- **Validity is what large blocks pay for.** hit degrades gently with block size (−12%p from blk1 to blk64) but validity collapses (−44%p). Within a block, first-hitting reveals tokens in *random* order, so in a large block a token is often committed far from any decided neighbor and its neighbors must then match it post hoc; at blk1 generation is strictly left-to-right and every step conditions on its immediate predecessor. Edge-validity is exactly the local-consistency property this random reveal order breaks.
- **The blk2–8 plateau, then the cliff.** blk2–8 are statistically close (valid^hit 0.78–0.82, with blk4 the best of the three); from blk16 onward every consistency metric drops sharply (valid^hit 0.60 → 0.45). If a genuinely block-parallel model is wanted, blk4 remains the sweet spot, matching the predecessor report's choice.
- **blk4 reproduces the previous report.** hit 0.943 / valid^hit 0.821 here vs 0.952 / 0.820 in `RESULTS_BD_prev.pdf` — within the ±1%p eval-sampling noise, on a different server, eval draw, and data order. The pipeline is stable end-to-end.
- **No horizon decay except at blk64.** Progressive block commitment keeps arrival flat (or even rising) with path length for blk1–32; only blk64 — where a whole 64-token canvas region is denoised against at most one committed block — shows decay at len 36+ (0.833). The full-canvas track's horizon-compounding failure does not reappear anywhere in the sweep.
- **Emergent length stays calibrated at every block size:** gen len tracks real len to within 0.1–0.5 tokens on average and len corr ≥ +0.977 for blk ≥ 2. blk1's lower len corr (+0.878) is a hit-rate artifact — with 99.7% arrival its lengths truncate exactly at dst, and the remaining variance is a handful of early-END misses — not a calibration failure (its mean length error is still 0.69).
- **Interpretation for the AR↔diffusion trade.** The sweep reproduces the BD3-LM paper's central trade-off in a planning domain: block size buys within-block parallelism at a likelihood/consistency cost. On this graph-constrained task the cost lands almost entirely on edge validity. Caveat from the code audit: this implementation omits bd3lms' clipped noise schedules (variance reduction that matters most at small block sizes), so small-block results here are, if anything, *underestimates* of the original recipe — which only strengthens the small-block conclusion. Within-sweep comparisons are unaffected (identical training conditions at all sizes).

4. Failure anatomy — where the length calibration comes from, what a miss is, and why validity falls

The measurements in this section are post-hoc diagnostics run on the saved checkpoints with fresh 400-pair draws (no refinement anywhere); with n_miss = 21–49 per run, subgroup percentages carry ~±10%p sampling noise.

4.1 The high len corr is mostly dst-truncation geometry, not END prediction

Generation length is decided by **where the dst token is emitted** for hit paths (the path is truncated at dst) and by **where END is emitted** only for the ~5–12% of miss paths. Splitting the correlation by subgroup:

block size	hit	len corr (all)	len corr (hit only)	len corr (miss only)	hit detour ratio gen/real
64	0.877	0.985	0.991	0.952	1.011
16	0.927	0.988	0.988	0.970	1.011
4	0.963	0.980	0.990	0.778	1.012

Hit paths reach dst with only a ~1.1% detour over the real (shortest) path, so their length is pinned by the OD pair's graph distance — the 0.99 correlation follows from geometry. This is why len corr survives at blk64 even as validity collapses: what large blocks break is local edge consistency, not the global "how far until dst" structure. END calibration is a real but secondary effect: it governs only the misses, where it is also good (miss-only corr 0.95–0.97 at large blocks; miss mean lengths match real means) — without it, misses would run to the canvas cap and drag the overall correlation down.

4.2 A miss is a near-miss: the model reaches the destination's doorstep

A **miss** = the model emitted END (declared termination) without ever emitting the exact dst vertex; in this all-raw report no post-processing touches it and it counts as an arrival failure. Measuring the graph hop distance from each miss path's final vertex to dst:

block size	n_miss	median hops to dst	mean	≤1 hop	≤2 hops	≤5 hops
64	40	1	1.8	52%	92%	98%
16	39	2	3.3	33%	87%	95%
4	21	1	2.8	52%	81%	90%

Combining hit rate with the miss-endpoint distribution, **~99% of all blk64 paths end within 2 hops of dst**. A miss is a token-level identification failure at the very last step ("emit END instead of the exact dst id"), not a navigation failure — consistent with the length spec training END immediately after dst inside the same block. This is also why refine_es (cut at the first dst-adjacent vertex, append dst) could recover +3%p arrival in the predecessor report: half the misses already end dst-adjacent.

4.3 Why validity falls at large blocks: sparse local skips, amplified by a path-level metric

	blk64	blk16
--	-------	-------

invalid edges (edge-level)	3.6% (332 / 9,130)	2.2% (199 / 9,150)
paths containing ≥ 1 invalid edge	52%	33%
invalid edges per affected path	1.6	1.5
true hop distance of invalid pairs	median 3; 91% ≤ 3 hops	median 3; 87% ≤ 3 hops
location of invalid edges	block-interior 332, block-boundary 0	interior 188, boundary 11

- **Amplification, not collapse.** Per-edge accuracy is still 96.4% at blk64; with ~ 23 edges per path, $P(\text{all edges valid}) \approx 0.964^{23} \approx 0.43$ — matching the observed 0.50 valid rate. The path-level all-or-nothing metric turns a 3.6% edge error rate into a 50% path failure rate.
- **The errors live strictly inside blocks.** Zero invalid edges at block boundaries (where the previous block is fully committed and the transition is effectively autoregressive). The cause is first-hitting's *random* within-block reveal order: a position revealed between two already-committed non-adjacent anchors faces a two-sided constrained-infilling conditional, combinatorially harder than the left-to-right case and less accurately learned — the error rate therefore grows with block size.
- **Skips are local.** Invalid pairs are truly 2–3 hops apart (median 3): the model drops a bridge vertex, it does not teleport.

4.4 Implications — remediation outlook

- **The model is not broken at large block sizes:** global structure (endpoint placement, length, near-shortest routing) is intact; the defects are sparse (3.6% of edges), local (2–3 hop skips), and precisely locatable (adjacency check). This failure profile favors **post-processing over retraining**.
- **Local splice (proposed):** replace each invalid edge (u, v) with the shortest path $u \rightarrow v$ (median insertion: 2 vertices). Validity $\rightarrow 1.0$ by construction, endpoints untouched so hit is unchanged — at blk64 $\text{valid} \wedge \text{hit}$ would rise from 0.454 to ≈ 0.877 (= hit) at an average cost of $\sim +1.7$ tokens per path. Composable with the existing endpoint refinements (`refine_nx`) for a full $\text{valid}=1 \wedge \text{arrival}=1$ guarantee.
- **Training-side fixes are second-order:** capacity/epochs/clipped schedules can lower the edge error rate but any-order infilling remains intrinsically harder than AR (the reference paper's own likelihood-vs-block-size trend).
- **Sampler-side fixes change the sampling distribution** and need empirical validation: revealing left-to-right within a block (or adjacency-constrained logit masking, present in the prior project and deliberately removed) breaks the equivalence between first-hitting-with-uniform-choice and the trained diffusion reverse process — a learned model's conditionals are not order-consistent, and the prefix-conditioned patterns $L \rightarrow R$ relies on are rarely seen in random-masking training of large blocks.

5. Post-processing results — P1 (local splice), P3 (endpoint patch), P1+P3

Implementation of §4.4's outlook (`eval_bd_postproc.py`): for each checkpoint, 1,000 OD pairs are generated once (raw, no refinement), then the SAME generations are scored under three post-processors. **P1** replaces every invalid consecutive pair (u, v) with the shortest path $u \rightarrow v$; **P3** (= `refine_nx`) grafts a shortest path from a miss path's endpoint to dst; **P1+P3** composes both. Fresh evaluation draw (seed 1), so raw values differ from §1 within the $\pm 1\%$ sampling noise.

blk	variant	arrival	valid	valid $\wedge$ hit	EM	PC	DTW	len err	patch len
1	raw	0.998	0.935	0.935	0.836	0.874	0.036	0.64	—
	P1+P3	1.000	1.000	0.998	0.854	0.902	0.034	0.99	0.46
4	raw	0.958	0.822	0.794	0.737	0.773	0.040	0.46	—
	P1+P3	1.000	1.000	0.958	0.764	0.841	0.043	1.33	1.14
8	raw	0.960	0.790	0.768	0.704	0.739	0.040	0.42	—
	P1+P3	1.000	1.000	0.960	0.757	0.838	0.044	1.23	1.20
16	raw	0.885	0.626	0.569	0.541	0.575	0.072	0.63	—
	P1+P3	1.000	1.000	0.885	0.588	0.708	0.072	2.58	2.59
32	raw	0.885	0.522	0.474	0.450	0.478	0.061	0.77	—
	P1+P3	1.000	1.000	0.885	0.558	0.686	0.060	2.14	2.06
64	raw	0.886	0.517	0.465	0.423	0.458	0.055	0.81	—
	P1+P3	1.000	1.000	0.886	0.569	0.695	0.056	2.01	2.01

Full per-variant rows (raw / P1 / P3 / P1+P3 separately) in `sets_res/BD_mask_blk*_postproc.res`. P1 alone gives the $\text{valid} \wedge \text{hit}$ numbers above (P3 does not change $\text{valid} \wedge \text{hit}$; hit is pre-refine); P3 alone gives $\text{arrival} = 1.0$ at raw validity.

- **§4.4's prediction is confirmed almost exactly:** at blk64, $\text{valid} \wedge \text{hit}$ 0.465 $\rightarrow$ 0.886 (= hit), at an average patch cost of 1.75 tokens; unfixable invalid pairs were zero across all runs.

- **P1 improves EM/PC too** at large blocks (blk64 EM 0.423 → 0.588, PC 0.458 → 0.711): the inserted bridges are real route segments that overlap the reference paths — the patch does not just legalize the path, it moves it toward the true route.
- **P1+P3 gives the structural guarantee $\text{valid} = 1 \wedge \text{arrival} = 1$** at every block size, with DTW essentially unchanged and len err +0.3–2.0 tokens. Under this post-processing the usable-path rate equals the raw hit rate (0.998 at blk1, 0.886 at blk64) — post-processing fully absorbs the large-block validity cost, leaving hit as the only remaining block-size penalty.

6. Relation to the previous report (RESULTS_BD_prev.pdf)

- The predecessor evaluated blk4 only, with refine variants: mask raw 0.952 arrival / 0.820 valid $\wedge$ hit, refine_es +0.03 arrival, refine_nx arrival 1.0 by construction. This sweep drops refinement entirely (user request) and varies block size instead.
- The predecessor's conclusion "refinement is nearly obsolete for the mask kernel" gets stronger here: at blk1, raw arrival is 0.997 — refinement would have $\leq 0.3\%$ of headroom.
- The graph (uniform-state CTMC) kernel was not part of this sweep (user decision: mask only). The queued graph work from the previous report (d = 0.05 retrain, respaced block sampling) remains open.

Artifacts

- Checkpoints: `sets_model/BD_porto_v3_normal_mask_blk{1,2,4,8,16,32,64}_base_bd.pth` (+ `_epoch_0` copies), server `143.248.84.179:/home/aailab/wp03052/Synthetic-Data/Block-Diffusion-0D-Planning`
- Results: `sets_res/BD_porto_v3_normal_mask_blk*_base_bd.res` (raw + refine_nx + refine_es variants stored; only raw reported here), post-processing comparison `sets_res/BD_mask_blk*_postproc.res` (`eval_bd_postproc.py`) + per-path EM/PC records under `sets_res/em_pc/`
- Training logs: `sets_log/bd_mask_blk*.log` , `sets_model/*_bd_train_log.txt`
- Code: `models_seq/bd_models.py` , `models_seq/bd_trainer.py` , `main_bd.py` , `train_porto_bd.sh` ; launcher: tmux sessions `bd_gpu{0..3}` (one GPU per run, ≤ 1 run/GPU per the utilization policy)
- Reference audit: `pdfs/BD_CODE_VERIFICATION_K0.pdf` / `_EN.pdf`